Pinterest Goku TSDB Architecture Breakdown

In-Memory C++ Time Series Engine, Gorilla Encoding & Roaring Bitmaps

TIME SERIES ENGINE

TELEMETRY SCALING

1. The Telemetry Challenge at Pinterest

Pinterest monitors millions of time series across thousands of microservices, host servers, and mobile recommendation pipelines, ingesting billions of metric datapoints daily.

The Downfall of OpenTSDB and HBase

Historically, Pinterest ran OpenTSDB on Apache HBase/HDFS. At production scale, HBase suffered catastrophic JVM GC freezes, heavy compaction I/O write amplification, and query aggregation latencies exceeding 5 to 10 seconds. Goku was designed in C++ to store 24 hours of telemetry purely in RAM.

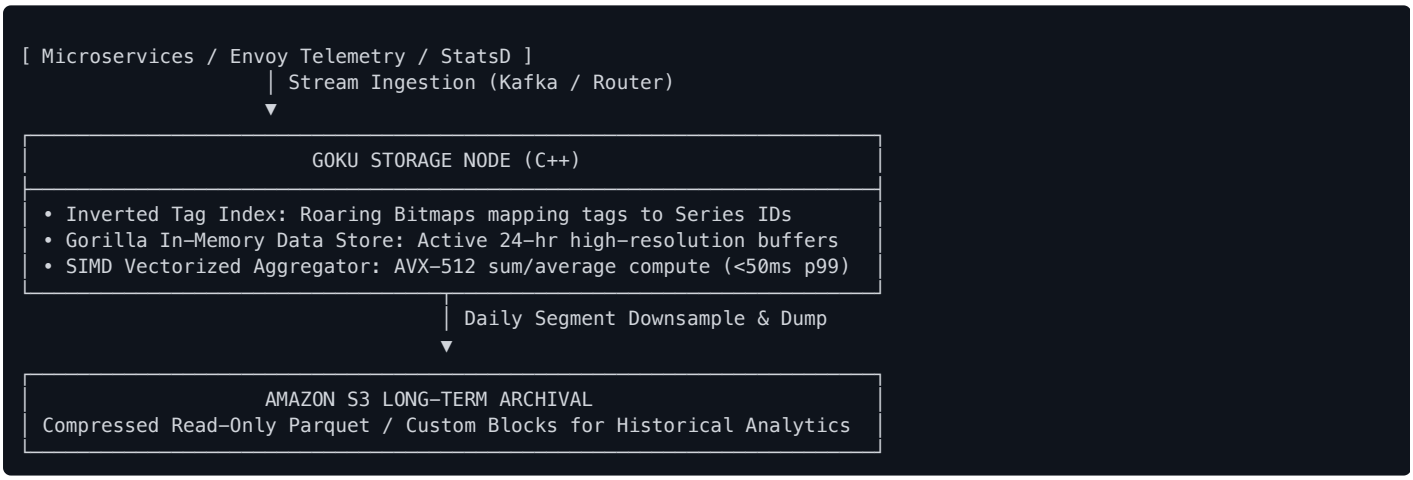
2. Gorilla Compression: Achieving 12x Memory Reduction

Storing billions of telemetry points in RAM is made economically viable through Facebook Gorilla compression algorithms:

```
1. Timestamp Delta-of-Delta Compression:
   • Compute delta D = T_i - T_(i-1). Compute delta-of-delta D' = D_i - D_(i-1).
   • If D' == 0: Store single bit '0'!
   • If -63 <= D' <= 64: Store '10' followed by 7 bits.
   • Slashes 64-bit timestamps down to an average of 1.37 bits per point!

2. Floating-Point XOR Compression:
   • Compute bitwise XOR: X = Float_i XOR Float_(i-1).
   • If X == 0: Store single bit '0' (Value unchanged).
   • If X != 0: Strip leading & trailing zeros, store variable-length bits.
   • Slashes 64-bit doubles down to an average of 1.3 bytes per value (83% savings)!
```

3. Goku In-Memory System Topology



4. Multi-Dimensional Tag Filtering with Roaring Bitmaps

In modern observability, metrics are filtered by arbitrary key-value tags: `metric=api.latency AND host=srv-10 AND env=prod`:

- **Dense Series ID Mapping:** Each distinct metric series is assigned an internal, dense 32-bit integer `SeriesID`.
- **Roaring Bitmap Inverted Index:** Every tag (e.g. `env=prod`) maps to a Roaring Bitmap containing all matching SeriesIDs.
- **Sub-Millisecond Bitwise Intersections:** Complex multi-tag queries execute via hardware-accelerated bitwise `AND` and `OR` instructions, resolving matching series in under 1 millisecond.

5. SIMD Vectorized Query Aggregation Engine

Calculating aggregates across thousands of time series is compute-intensive:

AVX-512 Vector Execution
Goku utilizes SIMD vector instructions to compute concurrent sums, averages, and percentiles over unpacked float slices directly in CPU L1/L2 caches.

Lock-Free Append Concurrency
Ingestion workers append points into ring buffers, publishing lengths via atomic memory barriers. Queries scan data lock-free without thread mutex contention.

6. Architectural Comparison: OpenTSDB vs. Pinterest Goku

Architectural Vector	OpenTSDB on Apache HBase	Pinterest Goku TSDB
Runtime Environment	Java JVM on HDFS / HBase	**Bare-Metal C++ (Zero Garbage Collection)**
Compression Efficiency	Snappy / Gzip (~4:1 ratio)	**Gorilla Delta-of-Delta + XOR Float (12:1)**
Dashboard Query p99	3,000ms – 10,000ms	**Sub-50ms p99 (100x faster!)**
Infrastructure Footprint	Hundreds of heavy HBase servers	**75% Server Fleet Reduction**

7. Two-Tiered Storage & S3 Block Persistence

Telemetry value follows a steep decay curve: 95% of queries hit data less than 12 hours old. Goku aligns storage economics accordingly:

- **Hot RAM Tier (0 to 24 Hours):** Uncompressed Gorilla buffers in RAM servicing live alerting checks and interactive dashboard browsing in sub-50ms.
- **Cold Object Storage Tier (24 Hours to 1 Year):** Background threads roll up 10-second data into 5-minute averages, packing segments into columnar blocks uploaded to Amazon S3 for historical trending queries.

8. Key Architectural Takeaways for Observability Systems

Core Principles Distilled from Pinterest Goku TSDB

- **In-Memory TSDBs Beat Disk Engines for Dashboards:** Storing live operational metrics in RAM eliminates disk I/O bottlenecks and delivers sub-50ms queries for on-call engineers.
- **Gorilla Compression Makes RAM Economical:** Delta-of-delta timestamp encoding combined with floating-point XOR packing achieves 12x compression, making 24 hours of RAM retention cost-effective.
- **Accelerate Multi-Tag Lookups with Bitmaps:** Roaring Bitmaps transform complex multidimensional metric queries into lightning-fast bitwise CPU operations.
- **Tier Data by Temporal Utility:** Retain raw high-resolution metrics in memory for 24 hours, downsample older data, and archive to inexpensive cloud object storage (S3).